

Q1 — Is the Atomspace ready to store the JSON shapes proposed in §2.2, §4.2, §7.1?

Partially. The file-system layer can support it, but key directories and the join document are missing.

What exists now:

- sample_data/atoms/performance/ — 100+ JSONs with ad_id, ROAS, CTR, spend, actions, purchase_roas
- sample_data/atoms/creative/ — 100+ JSONs with ad_id, headlines, bodies, images[].description (free-text, NOT controlled vocab)
- No commerce/, day/, brand/, or patterns/ directories exist

What to do (no schema redesign needed):

- Add directories: sample_data/atoms/commerce/, sample_data/atoms/day/, sample_data/atoms/brand/, sample_data/atoms/patterns/
- Write a sample_data/atoms/ad_index/<ad_id>.json with the cross-refs from §2.2
- Write learned patterns to sample_data/atoms/patterns/<pattern_id>.json so they persist across runs

No database required. The flat-file model is fine for the current data volume (~200 ads).

Q2 — Vision-LLM at scale? Anthropic API vs local? Cost for ~200 ads?

Two separate models for two separate jobs. Image generation already uses gpt-image-2. The controlled-vocab re-description step needs a vision/analysis model (gpt-4o-mini).

Image generation (already working):

- webapp/gen_image.py and webapp/server.js both call gpt-image-2 — this is correct and intentional. Cost for low quality image (1024 by 1024) = \$0.006/image × 200 images = ~~200 ads × 1.5 avg) = **~~\$1.2 total, Cost for high quality image (1024 by 1024) = \$0.211/image × 200 images = ~~200 ads × 1.5 avg) = **~~\$42.2 total

Vision-LLM for controlled vocab re-description (the new step):

- This is a separate task — reading each existing ad image and tagging it with the \$6 taxonomy (hook_type, visual_style, setting, etc.)

- Use gpt-4o-mini vision (already have the OpenAI client). This is an analysis model, not gpt-image-2 which is a generation model.
- Cost: gpt-4o-mini vision $\approx \$0.001/\text{image} \times 200 \text{ images}$ (~~$200 \text{ ads} \times 1.5 \text{ avg}$~~) = $**\$0.20$ total, one-off**
- Build webapp/describe_creatives.py — reads each sample_data/atoms/creative/*.json, sends the existing images[].url to gpt-4o-mini with the §6 taxonomy as a JSON schema, writes back the structured fields alongside the original description (kept as description_raw)

Q3 — Are the §5.2 confidence weights implementable in PLN, or does PLN impose a different combination?

PLN does not impose a formula. I computed confidence in Python and pass a single number into (stv s c). The weighted-sum formula is fully implementable.

What exists now at webapp/pln_query.py:

```
c_sample    = n / (n + 10)
c_spend     = min(1.0, total_spend / 5000)
c_consistency = max(0.0, 1.0 - stdev_s / max(mean_s, 0.001))
c_counter   = 1.0 - counter_n / (n + counter_n + 1)
conf_subst  = c_sample * c_spend * c_consistency * c_counter # MULTIPLICATIVE
```

This is multiplicative, not the weighted sum from §5.2. The lib_pln.metta Modus Ponens operator ($| \sim$) at repos/OmegaClaw-Core/lib_pln.metta only consumes the final c value — it doesn't know how we derived it.

To implement §5.2: We have to replace conf_subst with:

```
# Add recency_factor using date_start from performance atoms
age_days = (today - parse(p.get("date_start", today))).days
recency_factor = math.exp(-math.log(2) * age_days / HALF_LIFE_DAYS)

confidence_static = (
    w1 * sample_size_factor + # 1 - exp(-n/10)
```

```

w2 * spend_factor +      # min(1, total_spend/SPEND_REF)
w3 * consistency_factor + # 1 - CV
w4 * recency_factor +
w5 * counter_evidence_factor
) / (w1 + w2 + w3 + w4 + w5)

```

PLN is transparent to this change, I just pass the new confidence_static as the c in stv.

Q4 — Counter-evidence mining: clean way to query "condition C held but outcome failed" in MeTTa?

MeTTa cannot do threshold queries efficiently. It's done in Python, I would recommend to keep it there.

MeTTa limitation: The match &self operator in MeTTa can retrieve atoms by pattern, but comparing a numeric value against a dynamic threshold requires arithmetic evaluation (!) which makes it significantly more complex than a Python list comprehension. There's no SQL-style WHERE strength < 0.3.

Data shape recommendation: Keep OutcomeStv[ad_id][outcome] = (strength, confidence) in Python. For the persisted pattern atoms on disk, write counter_evidence: [ad_ids] into the pattern JSON (already done in counter_ads_out at line 486). This is the right architecture.

Q5 — Brand-atom ingest: Python script (per ADR 0003) or inside webapp/LLM service?

Standalone Python script, same pattern as pln_query.py. ADR 0003 doesn't exist in the repo yet — I should create it.

Current pattern in the repo: all data processing is in Python subprocess scripts invoked by server.js via runPython(). The brand atom is a **one-time/periodic operation** (not per-request), so it belongs as a standalone script, not wired into a live API endpoint.

Implementation:

- webapp/build_brand_atom.py — scrapes stoneybracelets.nl, extracts product catalog via HTML parsing, colors via CSS, copy samples from product pages, writes sample_data/atoms/brand/stoney.json

- webapp/parse_brand_pdf.py — if Robbin uploads a PDF brand book, parses it with pdfplumber and Claude Haiku/GPT-4o-mini, merges into the same brand JSON
- No new server.js API endpoint needed — this is a dev/admin operation that produces a static file consumed at generation time

The brand atom file is then loaded by pln_query.py and injected into the prompt construction in server.js's buildEditPrompt function.

Q6 — Where does "update pattern confidence on new performance" live?

Separate reconciliation script. The feedback scaffold exists in server.js but is currently write-only.

What exists now at webapp/server.js: /api/feedback writes {pattern_name, justification_patterns, observed_outcome, verdict, ts} to feedback_log.jsonl. The log is never read back — it's a dead end.

Two things missing:

1. Generated ads don't record pattern_provenance — which pattern_ids and confidences justified them
2. No script reads new performance back and updates pattern confidence

Architecture:

- When generating ads: write generated_ads.jsonl with {ad_id, pattern_provenance: [{pattern_id, confidence_at_gen_time}], generated_at}
- New script webapp/reconcile_patterns.py:
 - Reads generated_ads.jsonl
 - When a new performance atom appears for a generated ad_id, load the pattern JSON from sample_data/atoms/patterns/
 - If outcome matched prediction → increment evidence list, recalculate confidence
 - If outcome failed → add to counter_evidence list, recalculate confidence
 - Write updated pattern JSON back to disk
- I have to run this script manually for now after importing new Meta performance data; later wire to a cron or webhook

Keep it separate from performance import — the import script's job is to write atoms, the reconciliation script's job is to read them and update patterns.

Q7 — Recency half-life: is 180 days realistic for Stoney's data volume?

180 days is too long for jewelry/fashion. Start with 90 days general, 45 days for seasonal patterns.

What we can see from the data: performance atoms have date_start from 2026-01-28 through 2026-04-28 — about 3 months of data visible. Many are "synthetic": true, so actual historical depth is unknown.

Reasoning:

- Jewelry/lifestyle accessories respond to seasonality (Moederdag, Christmas, Valentine's) on 4–6 week cycles. A pattern learned from Moederdag 2025 is still relevant for Moederdag 2026 but not for a random July campaign.
- With ~20-40 new ads per month (estimate), at 180 days you'd have ~80-160 ads in the recency window — workable.
- At 90 days you'd have ~60-80 ads — still enough for statistically meaningful patterns with $n \geq 5$.

Recommendation: Configure HALF_LIFE_DAYS = 90 as an environment variable. Add a seasonal flag to patterns — seasonal patterns (hook=gifting, context=christmas/moederdag) use 45-day half-life; evergreen patterns use 90. Tune after first learning pass reveals actual data age distribution.

Q8 — Promotion threshold (confidence ≥ 0.65 , $n \geq 8$): acceptable or would you set differently?

Short answer: $n \geq 8$ is too strict for a first pass. Start at $n \geq 5$, confidence ≥ 0.50 . Make it configurable.

What the current code reveals: $c_sample = n / (n + 10)$ gives:

- $n=5 \rightarrow 0.33$
- $n=8 \rightarrow 0.44$
- $n=15 \rightarrow 0.60$
- $n=30 \rightarrow 0.75$

With a multiplicative formula (current) or weighted sum (proposed), hitting confidence ≥ 0.65 requires roughly $n \geq 15-20$ supporting ads. With ~ 200 ads across many condition combinations, most patterns will have $n=3-8$. Setting the bar at $n \geq 8 + 0.65$ confidence on a first pass may produce **zero promotable patterns**, which gives no improvement and no basis for tuning.

Recommendation:

```
PROMOTION_THRESHOLD_CONFIDENCE = float(os.environ.get("PROMOTION_CONF", "0.50"))
```

```
PROMOTION_THRESHOLD_N = int(os.environ.get("PROMOTION_N", "5"))
```

Start at $n \geq 5$, confidence ≥ 0.50 for the first learning pass. Once you see the distribution, raise it. The important thing is the threshold gates what goes into **generation** — sub-threshold hypotheses should still be visible in the UI so Robbin can see what the system is learning.